AWS Advanced Architecture — Scenario-Based Interview Prep

DevOps · SRE · Cloud Engineer · Mid to Senior Level

> **Focus:** Cross-account access · Cross-region architectures · Hybrid connectivity · Private/public access patterns · Service-to-service security · HA & failover · Security boundaries
>
> Every question is production-grade, architecture-heavy, and trade-off driven.

Table of Contents

#	Scenario	Domain
1	Multi-account VPC-to-VPC secure communication	Cross-account · VPC · PrivateLink
2	App in Region A accessing a database in Region B with low latency	Cross-region · Aurora · Latency
3	Migrating on-prem workloads to AWS with zero-downtime cutover	Hybrid · Direct Connect · VPN
4	Shared services VPC accessed by 50+ spoke VPCs securely	Transit Gateway · PrivateLink · Scale
5	Private API Gateway accessible only from on-prem and specific VPCs	Private Access · API GW · Hybrid
6	Multi-region active-active architecture with global routing	HA · Global Accelerator · Route 53
7	Cross-account S3 data pipeline with strict security boundaries	Cross-account · IAM · S3
8	Microservices across accounts communicating without internet exposure	PrivateLink · Multi-account · EKS
9	Direct Connect failover to VPN without dropping sessions	Hybrid · BGP · HA
10	Multi-account security boundary with centralised egress inspection	Network Firewall · TGW · Security
11	Database in a locked-down account accessed by Lambda in another account	Cross-account · RDS Proxy · Lambda
12	Active-passive DR across regions with sub-5-minute RTO	DR · RTO · Pilot Light

Scenario 1 — Multi-Account VPC-to-VPC Secure Communication

Scenario

Your company has adopted AWS Organizations with separate AWS accounts for Production, Staging, and a Shared Services account. A microservice running in **Account A (Production VPC: 10.0.0.0/16)** must communicate with a billing API running in **Account B (Shared Services VPC: 10.1.0.0/16)** over a private network.

Constraints:

- No traffic should traverse the internet at any point.
- Account B's VPC must not be fully exposed to Account A — only the billing API service should be reachable.
- The architecture must scale to 20+ accounts needing access to billing API without Account B managing 20 peering connections.
- Latency must be under 5ms (both VPCs are in `ap-south-1`).

Solution

Use **AWS PrivateLink (VPC Endpoint Service)** to expose only the billing API from Account B, and create **Interface VPC Endpoints** in each consumer account's VPC.

Architecture:

```
Account A (Production)                    Account B (Shared Services)

┌──────────────────────────┐              ┌──────────────────────────────┐
│  VPC: 10.0.0.0/16        │              │  VPC: 10.1.0.0/16            │
│                          │              │                              │
│  ┌──────────────────┐    │              │  ┌────────────────────────┐  │
│  │  Microservice    │    │  PrivateLink │  │  NLB (billing-nlb)     │  │
│  │  (EC2/ECS)       │──→ │──────────────│→ │  ↓                     │  │
│  └──────────────────┘    │              │  │  Billing API (ECS/EC2) │  │
│          │               │              │  └────────────────────────┘  │
│          ▼               │              │                              │
│  ┌──────────────────┐    │              │  VPC Endpoint Service        │
│  │  Interface VPC   │    │              │  (com.amazonaws.vpce.        │
│  │  Endpoint        │    │              │   ap-south-1.vpce-svc-xxx)   │
│  │  (ENI: 10.0.x.x) │    │              └──────────────────────────────┘
│  └──────────────────┘    │
└──────────────────────────┘
```

Step-by-Step Design:

Step 1 — Account B: Set up the NLB and Endpoint Service

The billing API must be fronted by a **Network Load Balancer** (NLB). PrivateLink requires an NLB or Gateway Load Balancer as the backend.

```
# Create NLB in Account B's VPC targeting billing API instances
# NLB listener: TCP 443 (or whatever the API port is)
```

```
# Target group: billing API EC2/ECS instances on port 8080
```

Register the NLB as a VPC Endpoint Service:

```
aws ec2 create-vpc-endpoint-service-configuration \
  --network-load-balancer-arns arn:aws:elasticloadbalancing:ap-south-
1:ACCOUNT_B:loadbalancer/net/billing-nlb/xxx \
  --acceptance-required true \
  --region ap-south-1
```

`--acceptance-required true` means Account B explicitly approves each consumer account. This is the security gate.

Step 2 — Account B: Grant Account A permission to connect

```
aws ec2 modify-vpc-endpoint-service-permissions \
  --service-id vpce-svc-xxxxxxxxxxxxxxxxx \
  --add-allowed-principals arn:aws:iam::ACCOUNT_A_ID:root
```

This whitelists Account A's entire account. For tighter control, specify a specific IAM role ARN instead of `root`.

Step 3 — Account A: Create an Interface VPC Endpoint

```
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-xxxxxxxx \
  --vpc-endpoint-type Interface \
  --service-name com.amazonaws.vpce.ap-south-1.vpce-svc-xxxxxxxxxxxxxxxxx
\
  --subnet-ids subnet-xxxxxxxx subnet-yyyyyyyy \
  --security-group-ids sg-xxxxxxxx \
  --region ap-south-1
```

This creates ENIs in Account A's subnets with private IPs (10.0.x.x range). The microservice calls the billing API using either:

- The endpoint DNS name (e.g., `vpce-xxx.vpce-svc-xxx.ap-south-1.vpce.amazonaws.com`)
- A Route 53 private hosted zone record aliasing to `billing.internal.company.com`

Step 4 — Security Group on the Interface Endpoint

The endpoint's SG must allow inbound from the microservice's SG on the API port:

```
Endpoint SG Inbound:
  Port: 443
```

```
    Source: sg-microservice (Account A)
```

Step 5 — No CIDR overlap concern

Because PrivateLink works at the service level (not IP routing), **CIDR overlap between Account A and Account B doesn't matter**. The endpoint gets an IP from Account A's CIDR space — it doesn't route to 10.1.x.x.

Scaling to 20+ accounts:

Each new consumer account follows Steps 3–4. Account B only manages a single Endpoint Service — not 20 peering connections. Account B maintains the approved principals list.

Why This Approach

Requirement	Why PrivateLink wins
No internet exposure	Traffic never leaves AWS backbone
No full VPC exposure	Only the NLB/service is accessible — not the entire VPC
Scales to many consumers	One Endpoint Service, N consumers
CIDR overlap safe	Not affected by overlapping address spaces
Under 5ms latency	Same-region PrivateLink is sub-millisecond overhead

---

Alternatives Considered

VPC Peering:

- Exposes the entire VPC to the peer — violates the "only billing API" constraint.
- Requires `N*(N-1)/2` connections for full mesh. At 20 accounts = 190 peering connections. Unmanageable.
- Does not work with overlapping CIDRs.

Transit Gateway with Resource Access Manager (RAM):

- Works for full network connectivity between accounts.
- Still exposes the entire VPC subnet routes — no service-level isolation.
- More appropriate when accounts need broad network access, not single-service access.

VPN over internet:

- Adds latency, encryption overhead, and internet dependency.
- Violates the "no internet" constraint.

Pitfalls / Mistakes

- **Using VPC Peering when PrivateLink is needed.** Peering exposes all subnets. Interviewers specifically test whether you distinguish "network-level" from "service-level" access.
- **Forgetting `acceptance-required`**. Leaving it as `false` means any account that knows the service name can connect without approval. Always enable for cross-account production services.
- **Not setting up private DNS for the endpoint.** Without a Route 53 alias or the VPC's `enableDnsHostnames` + `enableDnsSupport` flags set, the endpoint DNS doesn't resolve inside the VPC. Applications fail with DNS resolution errors.
- **NLB health check misconfiguration.** PrivateLink health is tied to the NLB target health. If targets are unhealthy, the endpoint appears connected but requests fail. Always monitor NLB target health, not just endpoint state.
- **CIDR overlap is not a concern, but candidates panic about it.** If asked "what if the CIDRs overlap?", the answer is: PrivateLink doesn't care. This is one of its key advantages over peering.

Scenario 2 — Application in Region A Accessing a Database in Region B with Low Latency

Scenario

Your SaaS application runs in `us-east-1`. Due to data residency laws (GDPR), European customer data must reside in `eu-west-1`. The application tier runs globally in both regions. The EU application tier (in `eu-west-1`) needs to write to a primary Aurora PostgreSQL database in `eu-west-1`. However, the US application tier frequently needs **read access** to that same EU database for cross-customer analytics with a latency requirement of under 50ms.

Constraints:

- EU data must never be stored in `us-east-1` permanently (GDPR compliance).
- US app tier reads must complete under 50ms p99.
- Write operations always go to `eu-west-1` primary.
- Architecture must survive an `eu-west-1` regional failure.
- Cost must be optimised — not all reads require the freshest data (analytics queries can tolerate up to 5-minute lag).

Solution

Use **Aurora Global Database** with a read replica cluster in `us-east-1`, combined with **Aurora cross-region read replicas** for analytics traffic. Route traffic using **Route 53 latency-based routing** and a **read/write split at the application layer**.

Architecture:

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Global Aurora Cluster                       │
│                                                                      │
│   eu-west-1 (Primary Cluster)          us-east-1 (Secondary)         │
│   ┌──────────────────────┐          ┌──────────────────────┐        │
│   │  Writer Instance     │──repl──►│  Read-only Cluster   │        │
```

```
│   │  + 2 Reader Instances │  ~1s lag   │  2 Reader Instances   │    │
│   └──────────────────────┘            └──────────────────────┘    │
│        ▲ writes                              ▲ analytics reads      │
│        │                                     │                     │
│   eu-west-1 App Tier               us-east-1 App Tier               │
└──────────────────────────────────────────────────────────────────┘

Route 53 (latency-based)
  ├── EU users → eu-west-1 writer endpoint (writes) / eu-west-1 reader
(fresh reads)
  └── US users → us-east-1 reader endpoint (analytics reads, ≤5min lag)
```

Step-by-Step Design:

Step 1 — Deploy Aurora Global Database

```
# Create the primary cluster in eu-west-1
aws rds create-global-cluster \
  --global-cluster-identifier my-global-db \
  --engine aurora-postgresql \
  --engine-version 15.2 \
  --deletion-protection

# Add secondary region (read-only cluster)
aws rds create-db-cluster \
  --db-cluster-identifier my-db-us-east-1 \
  --engine aurora-postgresql \
  --global-cluster-identifier my-global-db \
  --region us-east-1
```

Aurora Global Database uses **dedicated replication infrastructure** (not binlog replication) to replicate data at the storage layer — typically **under 1 second** lag between regions.

Step 2 — Application-layer read/write split

In the US application tier, configure the connection pool to use two endpoints:

- **Write endpoint:** Route 53 CNAME → Aurora primary cluster writer endpoint in `eu-west-1` (cross-region writes; ~80ms latency to eu-west-1 from us-east-1 is acceptable for write paths).
- **Read endpoint:** Route 53 CNAME → us-east-1 secondary cluster reader endpoint (sub-5ms, in-region).

```
# Application config
DB_WRITE_HOST = "writer.cluster-xxx.eu-west-1.rds.amazonaws.com"
DB_READ_HOST  = "reader.cluster-xxx.us-east-1.rds.amazonaws.com"

# ORM / connection pool routing:
# - All INSERT/UPDATE/DELETE → write pool
# - All analytics SELECT → read pool
```

Step 3 — Staleness management for analytics queries

Since the analytics read pool can have up to ~1 second of replication lag (often less), and the requirement allows up to 5 minutes for analytics, add a lag check:

```
-- Check Aurora replica lag before running analytics queries
SELECT EXTRACT(EPOCH FROM (now() - pg_last_xact_replay_timestamp())) AS
lag_seconds;
-- If lag_seconds > 300, route to eu-west-1 reader instead
```

Or use **Aurora's** `aurora_replica_read_consistency` session parameter to control read consistency level per query.

Step 4 — Regional Failover (eu-west-1 failure)

If `eu-west-1` goes down, Aurora Global Database supports **managed planned failover** and **unplanned failover**:

```
# Promote secondary to primary (takes ~1 minute)
aws rds failover-global-cluster \
  --global-cluster-identifier my-global-db \
  --target-db-cluster-identifier my-db-us-east-1
```

After promotion, update Route 53:

- Write CNAME now points to `us-east-1` writer endpoint.
- Write endpoint promoted to read-write.

GDPR note: For this failover scenario, confirm with legal — temporary writes to `us-east-1` during an `eu-west-1` outage may be permissible as a disaster recovery exception if data is migrated back when `eu-west-1` recovers.

Step 5 — Network path (Private)

Connections from the US app tier to the `eu-west-1` Aurora writer travel over the **AWS global backbone** (not the public internet) if the app runs in EC2/ECS within a VPC. Aurora is accessed via its VPC endpoint — traffic stays private across regions.

Why This Approach

Requirement	Solution Component	Reason
EU data residency	Primary in `eu-west-1` only	Data never permanently written to US region
<50ms US reads	In-region secondary cluster	Same-region reads: 1–5ms

Requirement	Solution Component	Reason
<1s replication lag	Aurora Global Database storage replication	Faster than binlog; ~100–500ms typical
Regional failover	Global Cluster managed failover	~60 second RTO; Aurora handles promotion
Analytics can tolerate 5min lag	Read from secondary; add lag check	No need to hit primary for non-critical reads

Alternatives Considered

Aurora Cross-Region Read Replica (non-Global):

- Uses binlog replication; lag can be several minutes under heavy write load.
- Global Database uses physical storage replication — fundamentally faster and more reliable.
- Global Database also enables managed cross-region failover; replicas do not.

DynamoDB Global Tables instead of Aurora:

- Fully managed multi-region active-active.
- But: GDPR compliance with DynamoDB Global Tables means data is replicated to all configured regions — this violates the "EU data stays in EU" requirement unless you carefully restrict regions.
- Also: not a relational database; may not fit the existing schema.

Application-layer caching (ElastiCache in us-east-1):

- Can satisfy analytics latency, but adds cache invalidation complexity.
- Cache is not a database — can't run arbitrary analytics queries.
- Works as a complementary layer, not a replacement for a read replica.

Read via VPN/Direct Connect to eu-west-1:

- Physical distance between `us-east-1` and `eu-west-1` is ~80ms RTT.
- Cannot achieve <50ms reads to a remote region regardless of connectivity. Physics wins.

Pitfalls / Mistakes

- **Trying to achieve <50ms reads cross-region.** You cannot beat the speed of light. `us-east-1` to `eu-west-1` is ~80ms RTT minimum. The only solution is a local replica.
- **Using Aurora Read Replica instead of Global Database.** Interviewers specifically test this. Replica lag on binlog-based replication can be seconds to minutes under load. Global Database is 10x more reliable.
- **Forgetting the GDPR write path.** Some candidates move the writer to `us-east-1` to reduce write latency. This immediately violates data residency. Writes must always go to `eu-west-1`.
- **No read/write split in the application.** Without explicit routing, all traffic goes to the writer, defeating the purpose of the read replica entirely. The application must be designed for this.
- **Failing to account for replication lag in query logic.** Analytics showing data from 30 seconds ago is fine. Analytics showing data from 3 hours ago (during a lag spike) is a business problem. Add lag

monitoring and alerting.

Scenario 3 — Hybrid Connectivity: On-Premises to AWS with Zero-Downtime Cutover

Scenario

A financial services company is migrating a core transaction processing system from their Mumbai on-premises data centre to AWS (`ap-south-1`). The system currently handles 50,000 transactions/day. The on-prem system must remain live during migration. During cutover, **no transactions can be lost and downtime must be under 2 minutes**.

Constraints:

- The on-prem DB (Oracle) must replicate to Aurora PostgreSQL in AWS before cutover.
- Applications on-prem and in AWS must communicate during the transition (hybrid period).
- After cutover, on-prem must still reach AWS for a 90-day parallel run period.
- The network path must be private — no data traverses the internet.
- BGP-based routing to allow gradual traffic shift.

Solution

Phase 1 — Establish Private Hybrid Connectivity (AWS Direct Connect + VPN Backup)

```
On-premises Data Centre (Mumbai)
        │
        │  AWS Direct Connect (1 Gbps dedicated connection)
        │  + Site-to-Site VPN (backup, over internet)
        ▼
AWS Direct Connect Location (Mumbai)
        │
        ▼
Virtual Private Gateway (VGW) → ap-south-1 VPC (10.0.0.0/16)
```

Why both DX and VPN:

- Direct Connect is the primary path: low latency (~2ms from Mumbai DC to ap-south-1), consistent bandwidth, no internet.
- VPN over internet is the automatic BGP failover: if DX fails during migration, VPN takes over within seconds. For a financial migration, single-path connectivity is unacceptable.

BGP configuration for path preference:

```
On-prem router BGP:
  - DX path: Local Preference = 200 (preferred)
  - VPN path: Local Preference = 100 (backup)
```

```
AWS side (VGW):
  - DX routes: no AS path prepending (preferred)
  - VPN routes: AS path prepend 3x (makes VPN less preferred)
```

Phase 2 — Data Replication Setup (Hybrid Period)

Use **AWS Database Migration Service (DMS)** with **AWS Schema Conversion Tool (SCT)** for Oracle → Aurora PostgreSQL:

```
On-prem Oracle DB ——— DMS Replication Instance ——► Aurora PostgreSQL
(ap-south-1)
                      (Full Load + CDC)
                      Continuous replication via DX
```

DMS in **Full Load + CDC (Change Data Capture)** mode:

1. Full load: migrates existing data (runs for hours/days depending on size).
2. CDC: continuously replicates changes (INSERT/UPDATE/DELETE) using Oracle LogMiner.
3. Replication lag monitored via DMS CloudWatch metrics: `CDCLatencySource`, `CDCLatencyTarget` — must be < 5 seconds before cutover.

Phase 3 — Application Communication During Hybrid Period

Both on-prem and AWS-side application components must communicate. Direct Connect enables this:

```
On-prem App Servers (192.168.1.0/24)
      │
      │ (Direct Connect — private routing)
      ▼
AWS VPC (10.0.0.0/16) — app tier in private subnets
      │
      ▼
Aurora PostgreSQL writer (10.0.2.100)
```

Route tables:

- On-prem router advertises `192.168.0.0/16` via BGP to AWS VGW.
- AWS VGW propagates this to VPC route tables.
- EC2 instances in AWS can reach on-prem private IPs directly — no NAT, no internet.

Phase 4 — Zero-Downtime Cutover (< 2 minutes)

```
Cutover Runbook:
T-30min: Verify DMS CDC lag < 2 seconds
T-10min: Notify all teams, freeze non-critical batch jobs
T-0:     1. Set on-prem app to read-only mode (drains in-flight
```

```
transactions)
         2. Wait for DMS CDC lag to reach 0 (all changes replicated)
         3. Pause DMS replication task
         4. Promote Aurora as primary (DMS task no longer needed)
         5. Update DNS (Route 53 or internal DNS) to point to Aurora
endpoint
         6. Restart application tier in AWS pointing to Aurora
         7. Route 53 weighted routing: 100% traffic to AWS app tier
T+2min:  Verify transactions processing in AWS
         Resume DMS in reverse (Aurora → Oracle) for 90-day parallel run
```

Phase 5 — 90-Day Parallel Run

Keep DX and VPN connectivity active. Reverse DMS replication (Aurora → Oracle) for audit/rollback capability. On-prem Oracle runs in read-only mode as warm standby.

Why This Approach

Requirement	Component	Reason
No internet data path	Direct Connect	Dedicated private fibre to AWS
Backup connectivity	Site-to-Site VPN with BGP	Automatic failover via BGP metric manipulation
Zero data loss	DMS CDC lag = 0 before cutover	All changes replicated before flip
< 2-minute cutover	DNS-based traffic shift	DNS TTL set to 30s pre-cutover for fast propagation
90-day parallel run	Maintain DX post-cutover	Sustained private connectivity for audit/fallback

---

Alternatives Considered

Direct Connect only (no VPN backup):

- Single point of failure. DX has physical layer dependencies (fibre, port). For a financial migration, this is a critical risk.
- A 4-hour DX outage during the migration window with no VPN fallback = aborted migration.

VPN only (no Direct Connect):

- VPN over internet introduces variable latency (~20–80ms vs ~2ms for DX in Mumbai).
- Bandwidth is capped and shared. DMS replication of large datasets over VPN is slow and risky.
- Not appropriate for financial data migration where consistency and speed matter.

AWS DataSync instead of DMS:

- DataSync is for file/object data (NFS, S3, FSx).

- DMS is for relational database replication with CDC. Not interchangeable.

Snowball for data transfer:

- Appropriate for bulk cold data (terabytes of archive data).
- Not appropriate for a live transactional database requiring continuous replication.

Pitfalls / Mistakes

- **Not setting DNS TTL low before cutover.** If DNS TTL is 300 seconds (5 minutes), traffic cutover takes 5 minutes even after you update the record — missing the 2-minute SLA. Set TTL to 30 seconds at least 24 hours before cutover.
- **Cutting over before DMS lag reaches zero.** Even 1 second of CDC lag means transactions committed on-prem are not yet in Aurora. Cutover with lag = data loss. Never skip this check.
- **Forgetting Oracle supplemental logging.** DMS CDC requires Oracle supplemental logging to be enabled. Without it, CDC cannot capture all changes. This is a pre-migration step that teams miss.
- **Not testing VPN failover before migration day.** Test the BGP failover weeks before. On migration day is not the time to discover the VPN failback takes 3 minutes instead of 30 seconds.
- **No rollback plan.** Every migration needs a clearly documented rollback trigger (e.g., "if error rate > 1% within 10 minutes of cutover, roll back"). Have the runbook ready. Reverse DMS task must be pre-staged, not set up after rollback is needed.

Scenario 4 — Shared Services VPC Accessed by 50+ Spoke VPCs Securely at Scale

Scenario

Your company runs an AWS Organisation with 50+ AWS accounts, each with their own VPC (all in `ap-south-1`). A central Shared Services VPC (in a dedicated account) hosts: an internal DNS resolver, a patch management server (WSUS/Ansible), a container image registry (ECR mirror), a secrets management service, and a centralised logging pipeline.

All spoke VPCs must access these services. New spoke VPCs are added monthly.

Constraints:

- Spoke VPCs must NOT be able to communicate with each other (lateral movement prevention).
- All spoke VPCs have potentially overlapping CIDRs (they were created independently — CIDR management was not enforced early on).
- Adding a new spoke account must be automated and take < 30 minutes.
- Cost must be controlled — don't pay for cross-VPC data transfer unnecessarily.
- Traffic from spoke VPCs to shared services must be inspectable/loggable.

Solution

Hybrid approach: AWS Transit Gateway (TGW) for DNS/management traffic + AWS PrivateLink for service-specific access.

The CIDR overlap constraint eliminates VPC Peering entirely. TGW with careful route table segmentation handles DNS/management, and PrivateLink handles service endpoints.

Architecture:

```
                    ┌──────────────────────────────────────────┐
                    │          Transit Gateway                 │
                    │   (ap-south-1, shared TGW via RAM)       │
                    │                                          │
                    │   Route Table: SPOKE-RT                  │
                    │   - Routes only to SharedSvc attachment│
                    │   - No spoke-to-spoke routes             │
                    │                                          │
                    │   Route Table: SHAREDSVC-RT              │
                    │   - Routes back to all spoke CIDRs       │
                    └──────────────────────────────────────────┘
                           │                  │
                ┌──────────▼──────┐        ┌──────▼──────────────┐
                │  Spoke VPC A    │        │  Spoke VPC B        │
                │  (10.0.0.0/16)  │        │  (10.0.0.0/16)      │
                │  ← overlapping  │        │  ← overlapping CIDR │
                └─────────────────┘        └─────────────────────┘
                           │
                           │ (Only to Shared Services — not to other
spokes)
                           ▼
                ┌──────────────────────────────────────────────────┐
                │          Shared Services VPC (10.255.0.0/16)     │
                │                                                  |
                │   ┌──────────────┐   ┌──────────────────────────┐   │
                │   │  Route 53    │   │  VPC Endpoint Services   │   │
                │   │  Resolver    │   │  (via PrivateLink NLBs)  │   │
                │   │  Inbound EP  │   │  - ECR Mirror            │   │
                │   └──────────────┘   │  - Secrets Service       │   │
                │                      │  - Logging Forwarder     |   |
                │   ┌──────────────┐   └──────────────────────────┘   │
                │   │  Patch Mgmt  │                                  |
                │   │  Servers     │                                  |
                │   └──────────────┘                                  |
                └──────────────────────────────────────────────────┘
```

Step 1 — Share the Transit Gateway via AWS RAM

```
aws ram create-resource-share \
  --name "SharedTGW" \
  --resource-arns arn:aws:ec2:ap-south-1:SHARED_ACCOUNT:transit-
gateway/tgw-xxxxxxxx \
  --principals "arn:aws:organizations::MASTER_ACCOUNT:organization/o-
xxxxxxxxxx"
```

Sharing at the Organisation level means all existing and future accounts automatically see the TGW — no manual per-account sharing needed.

Step 2 — TGW Route Table Segmentation (no spoke-to-spoke routing)

Create two route tables in the TGW:

```
SPOKE-RT (associated with all spoke VPC attachments):
  - 10.255.0.0/16 → SharedSvc TGW attachment
  - (No routes to other spokes — they don't exist here)
  - Blackhole for everything else

SHAREDSVC-RT (associated with Shared Services attachment):
  - 0.0.0.0/0 → SharedSvc VPC local
  - (Propagate routes from all spoke attachments — enables return traffic)
```

Crucially: spoke attachments are associated with `SPOKE-RT` but propagate routes into `SHAREDSVC-RT` only. This means:

- Spoke → SharedSvc: routed (SPOKE-RT has a route to 10.255.0.0/16)
- Spoke → Spoke: **not routed** (SPOKE-RT has no routes to other spoke CIDRs)
- SharedSvc → any Spoke: routed (SHAREDSVC-RT propagates all spoke CIDRs)

Handling Overlapping CIDRs on TGW:

TGW supports overlapping CIDRs across VPC attachments! Unlike VPC Peering, TGW uses **per-attachment routing** — each attachment has its own routing context. The TGW knows traffic for VPC-A's `10.0.0.0/16` goes to attachment `tgw-attach-aaa` and traffic for VPC-B's `10.0.0.0/16` goes to `tgw-attach-bbb`. Responses are routed back to the originating attachment.

However: The Shared Services VPC cannot initiate connections to overlapping-CIDR spokes (ambiguous routing). Only spoke → SharedSvc is unambiguous. For management/patch traffic that SharedSvc initiates to spokes, use **AWS Systems Manager (SSM)** — the SSM agent in spoke VPCs calls outbound to SSM endpoints, reversing the connection direction.

Step 3 — PrivateLink for Service Endpoints

For ECR mirror, logging, and secrets services — use PrivateLink (as in Scenario 1) so these are consumed over Interface VPC Endpoints. This means:

- No TGW data transfer charges for these (high-volume) services.
- Service-level access control per consumer.
- No routing required — endpoints get IPs from the consumer's own VPC CIDR.

Step 4 — Route 53 Resolver for DNS

Spoke VPCs need to resolve internal DNS names (e.g., `patch.internal.company.com` → Shared Services IP).

```
# Create Route 53 Resolver Inbound Endpoint in Shared Services VPC
aws route53resolver create-resolver-endpoint \
  --creator-request-id unique-id \
  --security-group-ids sg-xxxxxxxx \
  --direction INBOUND \
  --ip-addresses SubnetId=subnet-aaa,Ip=10.255.1.10 SubnetId=subnet-
bbb,Ip=10.255.2.10

# In each spoke account, create Resolver Forwarding Rule:
# "internal.company.com" → forward to 10.255.1.10, 10.255.2.10
# Share this Resolver Rule via RAM across the Organisation
```

Step 5 — Automate new spoke onboarding (< 30 min)

Use **AWS Service Catalog** or **Terraform with AWS Organizations event trigger**:

1. New account created → EventBridge event fires.
2. Lambda in the management account:
   - Accepts the TGW resource share invitation in the new account (via RAM).
   - Creates TGW attachment in the new account's VPC.
   - Associates attachment with `SPOKE-RT`.
   - Creates Interface VPC Endpoints for PrivateLink services.
   - Associates RAM-shared Route 53 Resolver Rule.
3. Total automation time: ~10–15 minutes.

Why This Approach

Constraint	Solution	Reason
Overlapping CIDRs	TGW (not peering)	TGW supports overlapping CIDRs across attachments
No spoke-to-spoke	TGW route table segmentation	Spoke-RT has no spoke CIDR routes
Service-level isolation (ECR, Secrets)	PrivateLink endpoints	Bypass TGW for high-volume; service-level control
Scale to 50+ accounts	RAM-shared TGW + automation	No manual per-account config; event-driven
Cost control	PrivateLink for high-volume services	TGW charges per GB; PrivateLink often cheaper at volume

---

Alternatives Considered

Full mesh VPC Peering:

- Fails immediately due to overlapping CIDRs. Not viable.

PrivateLink only (no TGW):

- PrivateLink doesn't handle DNS resolver traffic or patch management TCP sessions well (NLB fronting adds complexity for SSH/WinRM).
- TGW handles general TCP connectivity; PrivateLink handles specific service endpoints. Hybrid is correct.

Centralised Egress with TGW (all traffic through Shared Services):

- Some architectures route ALL internet egress through a central inspection VPC on TGW.
- Valid for security inspection use cases (adds AWS Network Firewall).
- Adds latency and TGW data transfer costs for internet traffic.
- Not required here since the focus is shared services access, not internet egress.

Pitfalls / Mistakes

- **Associating spoke attachments with both route tables.** Each attachment is associated with ONE route table. Propagation is how routes spread. Candidates confuse association (which table is used for routing FROM this attachment) with propagation (which table receives routes from this attachment).
- **Forgetting TGW data transfer costs.** TGW charges $0.02/GB processed. High-volume services like ECR (multi-GB image pulls) will generate significant TGW costs if routed through it. Use PrivateLink for data-heavy services.
- **Not testing overlapping CIDR routing.** Just because TGW supports it doesn't mean it works automatically. The return path from Shared Services to spoke VPCs relies on TGW knowing which attachment the traffic came from. Test this explicitly with two spokes sharing the same CIDR.
- **Missing the RAM auto-accept for new accounts.** If the new account must manually accept the RAM invitation, your automation breaks. Enable `auto-accept` at the Organisation level or include the acceptance call in your automation Lambda.

Scenario 5 — Private API Gateway Accessible Only from On-Premises and Specific VPCs

Scenario

Your team builds an internal API on **AWS API Gateway (REST)**. This API must be accessible from:

1. Your on-premises network (connected via Direct Connect).
2. Two specific VPCs in the same AWS account (VPC-Finance, VPC-HR).
3. **No other VPC, no public internet access whatsoever.**

Constraints:

- Access from the internet (even with API keys) is completely forbidden by your security policy.
- Each consumer (on-prem, VPC-Finance, VPC-HR) should have their access independently revocable.
- API must pass a WAF inspection layer.
- Latency from on-prem to API must be < 20ms (Direct Connect to ap-south-1).

Solution

Use **Private API Gateway** with a **VPC Endpoint (Interface)**, resource policy for access control, and **AWS WAF** on the API Gateway.

Architecture:

```
On-premises (192.168.0.0/16)
        │
        │  Direct Connect → VGW
        ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Shared Networking VPC (10.255.0.0/16)                               │
│                                                                      │
│   ┌──────────────────────────────────────────────┐                   │
│   │  Interface VPC Endpoint for API GW           │                   │
│   │  (com.amazonaws.ap-south-1.execute-api)      │                   │
│   │  ENI IPs: 10.255.1.10, 10.255.1.11           │                   │
│   └──────────────────────────────────────────────┘                   │
│                      │                                               │
│        Route 53 Private Hosted Zone:                                 │
│        *.execute-api.ap-south-1.amazonaws.com → VPC Endpoint         │
└──────────────────────┬──────────────────────────────────────────────┘
                       │  VPC Peering / TGW
           ┌───────────┴──────────┐
           ▼                      ▼
   VPC-Finance              VPC-HR
   (10.1.0.0/16)            (10.2.0.0/16)
           │                      │
           └──────────┬───────────┘
                      ▼
           ┌──────────────────────────────┐
           │  Private API Gateway         │
           │  + WAF Web ACL               │
           │  + Resource Policy           │
           │  (ap-south-1)                │
           └──────────────────────────────┘
```

Step 1 — Create a Private API Gateway

In the API Gateway console/CLI, set the endpoint type to `PRIVATE`:

```
aws apigateway create-rest-api \
  --name "internal-api" \
  --endpoint-configuration types=PRIVATE
```

A Private API Gateway is **only accessible via a VPC Endpoint**. It has no public DNS endpoint exposed to the internet.

Step 2 — Create the Interface VPC Endpoint

Create one VPC Endpoint for API Gateway in the Shared Networking VPC (which is accessible from on-prem via DX):

```
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-shared \
  --vpc-endpoint-type Interface \
  --service-name com.amazonaws.ap-south-1.execute-api \
  --subnet-ids subnet-private-a subnet-private-b \
  --security-group-ids sg-api-endpoint \
  --private-dns-enabled true
```

`--private-dns-enabled true` causes the endpoint to override the DNS resolution for `*.execute-api.ap-south-1.amazonaws.com` within the VPC. Any VPC that can reach this endpoint (via TGW or peering) and has this DNS resolution available will route API calls through the endpoint.

Step 3 — Give VPC-Finance and VPC-HR access to the endpoint

Option A: Peer VPC-Finance and VPC-HR to the Shared Networking VPC (or via TGW). The endpoint ENI is in Shared VPC's subnet — traffic from Finance/HR routes to it via the VPC peering connection.

Option B: Create separate VPC Endpoints for each VPC (more isolated, independently revocable).

For independent revocability, use **Option B** with separate endpoints. Set up the API Gateway resource policy to reference each VPC Endpoint ID:

Step 4 — API Gateway Resource Policy

Resource policy is how the Private API Gateway enforces which VPC Endpoints can invoke it:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": "arn:aws:execute-api:ap-south-1:ACCOUNT:api-id/*",
      "Condition": {
        "StringEquals": {
          "aws:sourceVpce": [
            "vpce-shared-networking",
            "vpce-finance",
            "vpce-hr"
          ]
        }
      }
    },
    {
```

```
        "Effect": "Deny",
        "Principal": "*",
        "Action": "execute-api:Invoke",
        "Resource": "arn:aws:execute-api:ap-south-1:ACCOUNT:api-id/*",
        "Condition": {
          "StringNotEquals": {
            "aws:sourceVpce": [
              "vpce-shared-networking",
              "vpce-finance",
              "vpce-hr"
            ]
          }
        }
      }
    ]
  }
```

The explicit **Deny for all other VPC endpoints** is critical — without it, any endpoint in the account could potentially call the API.

To revoke VPC-HR access: Remove `vpce-hr` from the allowed list and update the resource policy. No infrastructure change needed.

Step 5 — Attach WAF Web ACL

```
aws wafv2 associate-web-acl \
  --web-acl-arn arn:aws:wafv2:ap-south-1:ACCOUNT:regional/webacl/internal-
api-waf/xxx \
  --resource-arn arn:aws:apigateway:ap-south-1::/restapis/api-
id/stages/prod
```

WAF on Private API Gateway inspects all requests, even from private sources. This catches injection attacks, bot traffic (from compromised internal systems), and enforces rate limits per source IP.

Latency path for on-prem:

```
On-prem → DX (~2ms) → Shared VPC → VPC Endpoint ENI (~0.5ms) → API GW →
Lambda/Integration
Total network latency: ~3–5ms → well under 20ms requirement
```

Why This Approach

Requirement	Component	Reason
No internet access	Private API GW endpoint type	No public DNS; not resolvable externally

Requirement	Component	Reason
On-prem access	VPC Endpoint in DX-connected VPC	DX routes private traffic to VPC; endpoint accessible
Independent revocability	Per-consumer VPC Endpoint + Resource Policy	Remove endpoint ID from policy to revoke
WAF inspection	WAF Web ACL on API GW stage	Inspects all traffic regardless of source
<20ms latency	DX + same-region endpoint	Physical proximity; no internet hops

---

Alternatives Considered

Public API Gateway with IP allowlisting:

- The API would still be publicly resolvable. Even with WAF IP restriction, the endpoint is internet-exposed in DNS — violates "no internet access" policy.
- IP allowlisting on WAF is fragile (DX NAT IPs can change, multiple IPs per DX).

API hosted on EC2 behind ALB (internal):

- Valid alternative but loses API Gateway's managed features: auth, throttling, caching, logging, automatic scaling.
- Cross-account access is harder with an internal ALB.

Lambda Function URL with resource policy:

- Function URLs don't support `PRIVATE` endpoint type for cross-account/on-prem access.
- API Gateway is the right tool for this requirement.

Pitfalls / Mistakes

- **Creating a Private API but not updating the resource policy.** A Private API with no resource policy defaults to denying all access — even from the correct VPC Endpoint. You will get a 403 with no helpful error message. Always deploy resource policy alongside the API.
- **Enabling private DNS on the endpoint without understanding the implications.** With `private-dns-enabled=true`, ALL API Gateway calls from that VPC resolve to the endpoint — including calls to other teams' public APIs. Test carefully.
- **Forgetting the on-prem DNS resolution.** On-prem DNS servers don't know about AWS private hosted zones. You must configure DNS forwarding from on-prem DNS to Route 53 Resolver Inbound Endpoints for `execute-api.ap-south-1.amazonaws.com` to resolve to the VPC Endpoint IP.
- **Not accounting for endpoint-level security groups.** The VPC Endpoint has a security group. If it doesn't allow inbound HTTPS from the consumer's subnet, calls fail with a connection timeout — not a 403. This is a common source of confusion.

Scenario 6 — Multi-Region Active-Active Architecture with Global Routing

Scenario

You are designing a global e-commerce platform serving users in India and Europe. The platform must:

- Serve Indian users from `ap-south-1` (Mumbai) and European users from `eu-west-1` (Ireland).
- Both regions must be **fully active** — not active/passive.
- If one region becomes unhealthy, 100% of traffic must shift to the surviving region automatically within 30 seconds.
- User sessions must be maintained during regional failures (stateless app tier, shared session store).
- Database writes must be globally consistent (no split-brain).
- The solution must handle 100,000 requests/second peak.

Constraints:

- Session data must be accessible from both regions.
- Failover must be automatic — no manual DNS changes.
- Data must remain GDPR-compliant (EU user data residency).

Solution

AWS Global Accelerator for anycast global routing + **Aurora Global Database** for data layer + **ElastiCache Global Datastore** for sessions + **Route 53 Health Checks** for DNS-level failover.

Architecture:

```
Users (India)              Users (Europe)
       │                          │
       └──────────┬───────────────┘
                  ▼
      ┌──────────────────────────┐
      │   AWS Global Accelerator  │
      │   Anycast IPs: 75.2.x.x   │
      │   (2 static IPs globally) │
      └──────────────────────────┘
                  │
          ┌───────┴───────┐
          ▼               ▼
  ┌──────────────┐   ┌──────────────┐
  │  ap-south-1  │   │   eu-west-1  │
  │  ALB + ECS   │   │   ALB + ECS  │
  │  App Tier    │   │   App Tier   │
  │  (active)    │   │   (active)   │
  └──────────────┘   └──────────────┘
        │                  │
        │     ┌────────────┴──┐
        ▼     ▼               ▼
  ┌──────────────────────────────┐
  │        Aurora Global Database │
  │  Primary: eu-west-1 (writer)  │
  │  Secondary: ap-south-1 (reader) │
```

```
│  Cross-region replication: ~500ms lag   │
└─────────────────────────────────────────┘
       │                    │
┌──────▼────────────────────▼──────┐
│   ElastiCache Global Datastore   │
│   (Redis) — session storage      │
│   Primary: ap-south-1            │
│   Replica: eu-west-1             │
└──────────────────────────────────┘
```

Component Details:

Global Accelerator — the critical routing layer:

- Provides 2 static anycast IP addresses that are globally advertised from AWS edge nodes.
- User traffic enters the AWS backbone at the nearest edge point of presence — not at the regional ALB.
- Global Accelerator then routes traffic over the AWS backbone to the healthiest, nearest configured endpoint (ap-south-1 ALB or eu-west-1 ALB).
- **Health check:** every 10 seconds per endpoint. If an endpoint's health check fails 3 consecutive times, Global Accelerator removes it from routing within ~30 seconds.
- **Traffic dial:** Can shift 0–100% of traffic to each region, enabling gradual failover or canary releases.

```
aws globalaccelerator create-accelerator \
  --name ecommerce-global \
  --ip-address-type IPV4 \
  --enabled

# Add listener
aws globalaccelerator create-listener \
  --accelerator-arn arn:aws:globalaccelerator::ACCOUNT:accelerator/xxx \
  --protocol TCP \
  --port-ranges FromPort=443,ToPort=443

# Add endpoint groups (one per region)
aws globalaccelerator create-endpoint-group \
  --listener-arn arn:aws:globalaccelerator::ACCOUNT:listener/xxx \
  --endpoint-group-region ap-south-1 \
  --traffic-dial-percentage 50 \
  --endpoint-configurations EndpointId=arn:aws:elasticloadbalancing:ap-
south-1:...,Weight=100

aws globalaccelerator create-endpoint-group \
  --listener-arn arn:aws:globalaccelerator::ACCOUNT:listener/xxx \
  --endpoint-group-region eu-west-1 \
  --traffic-dial-percentage 50 \
  --endpoint-configurations EndpointId=arn:aws:elasticloadbalancing:eu-
west-1:...,Weight=100
```

Session Management — ElastiCache Global Datastore:

```
aws elasticache create-global-replication-group \
  --global-replication-group-id-suffix ecommerce-sessions \
  --primary-replication-group-id sessions-ap-south-1
# Add eu-west-1 as secondary
aws elasticache create-replication-group \
  --replication-group-id sessions-eu-west-1 \
  --global-replication-group-id global-ecommerce-sessions \
  --region eu-west-1
```

Replication lag: ~100ms between regions. App reads from local Redis; writes go to primary and replicate.

GDPR Compliance:

EU user data (PII) must not be stored in ap-south-1. Architecture handles this at the application layer:

- App identifies user's region via a `user_region` claim in the JWT/session token.
- EU users' PII is written to Aurora `eu-west-1` writer only.
- EU session data is tagged with `region=eu`; session routing logic reads from eu-west-1 Redis only for EU-flagged sessions.

Failover Scenario — ap-south-1 goes down:

1. Global Accelerator health check detects ap-south-1 ALB unhealthy (3 failed checks × 10s = ~30s).
2. Traffic dial for ap-south-1 endpoint group effectively drops to 0.
3. 100% traffic routed to eu-west-1 endpoint group.
4. EU users experience no change. Indian users now route to eu-west-1 (higher latency but functional).
5. Aurora Global Database: eu-west-1 is already the writer — no change needed.
6. ElastiCache: eu-west-1 replica is already available; Indian user sessions may have stale data if session was recently written in ap-south-1 and replication hadn't completed (~100ms window).

Why This Approach

Requirement	Component	Reason
Active-active routing	Global Accelerator	Anycast routing, health-based failover
<30s failover	Global Accelerator health checks	3×10s = 30s detection + immediate routing change
Shared sessions	ElastiCache Global Datastore	Sub-second replication; read local, write global
Global DB consistency	Aurora Global Database	Storage-layer replication, single writer
100k RPS	ECS Auto Scaling + ALB	Horizontal scaling per region

Requirement	Component	Reason
GDPR	App-layer routing by user region	EU PII stays in eu-west-1

Alternatives Considered

Route 53 Latency-Based Routing with Health Checks:

- DNS-based routing with TTL of 60s = minimum 60s failover time (often longer due to client DNS caching).
- Global Accelerator routes at the network layer — no DNS TTL limitation. Failover in 30s without DNS changes.
- Route 53 is a fallback option if Global Accelerator cost (~$18/month + data transfer) is a concern.

CloudFront as global entry point:

- CloudFront is optimised for HTTP caching at edge (static assets, cacheable API responses).
- Not appropriate for dynamic, stateful transactional API traffic requiring session affinity.
- Global Accelerator is the correct choice for non-cacheable, dynamic TCP traffic.

DynamoDB Global Tables instead of Aurora:

- Fully active-active writes in both regions.
- Trades ACID guarantees for availability — eventual consistency means potential conflict resolution.
- For an e-commerce platform with inventory and payment operations, ACID is mandatory. Aurora Global Database (single writer) maintains consistency at the cost of write latency from the secondary region.

Pitfalls / Mistakes

- **Confusing Global Accelerator with CloudFront.** They're both "global" but serve different purposes. CloudFront caches content at edges. Global Accelerator uses edge nodes only as entry points — all processing still happens in the AWS region. Interviewers test this distinction.
- **Active-active with a single database writer.** True active-active writes require conflict resolution (DynamoDB Global Tables, CockroachDB). Aurora Global Database has a single writer — technically it's active-active at the app tier but active-passive at the DB write tier. Acknowledge this trade-off explicitly.
- **Not accounting for session replication lag in failover.** If Indian users are active and ap-south-1 fails, their sessions are in the ap-south-1 Redis. The eu-west-1 replica may have sessions from up to 100ms ago. In practice this means users may need to re-login. If this is unacceptable, use write-through to both regions with synchronous confirmation (doubles write latency).
- **Forgetting that Global Accelerator endpoint group health checks need proper ALB health endpoints.** Global Accelerator checks your ALB, which checks your targets. A misconfigured target health check means Global Accelerator thinks the region is healthy even when the app is down.

Scenario 7 — Cross-Account S3 Data Pipeline with Strict Security Boundaries

Scenario

Your company runs a data platform: raw data lands in **Account A (Data Ingestion)**, is processed by a Spark job in **Account B (Data Processing)**, and final results are stored in **Account C (Data Warehouse)**. An analytics team in **Account D (Analytics)** needs read-only access to Account C's results.

Constraints:

- Account A must NOT be accessible by Account D (raw data is sensitive — PII).
- All data at rest must be encrypted with customer-managed KMS keys (each account manages its own key).
- The Spark job in Account B must read from Account A's S3 and write to Account C's S3.
- IAM permission boundaries must ensure Account B's Spark role cannot access Account D's data.
- All access must be logged and auditable.

Solution

Cross-account S3 bucket policies + IAM role chaining + KMS key policies per account.

Architecture:

```
Account A (Ingestion)          Account B (Processing)         Account C
(Warehouse)
┌──────────────────┐          ┌────────────────────┐
┌────────────────────┐
│  S3: raw-data    │          │  EMR/Glue Spark Job │         │  S3:
results        │
│  KMS Key A       │◄─────────│  IAM Role: spark-   │────────►│  KMS Key C
│
│  Bucket Policy:  │  Read    │  processor-role     │  Write  │  Bucket
Policy:     │
│  Allow Account B │  only    │                     │         │  Allow
Account B    │
│  spark role      │          │  Permission         │         │  (write
only)       │
└──────────────────┘          │  Boundary applied  │         │  Allow
Account D    │
                              └────────────────────┘         │  (read
only)        │

└────────────────────┘
                                                                     |
                                                            Account D
(Analytics)

┌────────────────────┐
                                                            │  Athena /
Redshift  │
                                                            │  IAM Role:
analyst  │
                                                            │  Cross-acct
```

```
assume  │

└──────────────────────┘
```

Step 1 — Account A: S3 Bucket Policy (allow Account B's Spark role to read)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAccountBSparkRead",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT_B:role/spark-processor-role"
      },
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::raw-data-bucket",
        "arn:aws:s3:::raw-data-bucket/*"
      ]
    }
  ]
}
```

Note: This policy **alone is not enough**. Account B's IAM role must also have a policy allowing it to access Account A's bucket. Both the bucket policy (resource-based) and the role policy (identity-based) must allow the action.

Step 2 — Account A: KMS Key Policy (allow Account B to decrypt)

```
{
  "Sid": "AllowAccountBDecrypt",
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::ACCOUNT_B:role/spark-processor-role"
  },
  "Action": ["kms:Decrypt", "kms:GenerateDataKey"],
  "Resource": "*"
}
```

Critical: Without this KMS key policy, the Spark job can read S3 objects but cannot decrypt them — it gets an `AccessDenied` on the KMS Decrypt call.

Step 3 — Account B: IAM Role for Spark with Permission Boundary

The Spark role's IAM policy:

```
{
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::raw-data-bucket",
        "arn:aws:s3:::raw-data-bucket/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:PutObjectAcl"],
      "Resource": "arn:aws:s3:::results-bucket/*"
    },
    {
      "Effect": "Allow",
      "Action": ["kms:Decrypt"],
      "Resource": "arn:aws:kms:ap-south-1:ACCOUNT_A:key/key-id-a"
    },
    {
      "Effect": "Allow",
      "Action": ["kms:GenerateDataKey", "kms:Encrypt"],
      "Resource": "arn:aws:kms:ap-south-1:ACCOUNT_C:key/key-id-c"
    }
  ]
}
```

Permission Boundary on the Spark role (prevents escalation to Account D's resources):

```
{
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::raw-data-bucket*",
        "arn:aws:s3:::results-bucket*"
      ]
    },
    {
      "Effect": "Deny",
      "Action": "*",
      "Resource": "arn:aws:s3:::analytics-bucket*"
    }
  ]
}
```

The permission boundary explicitly denies access to Account D's resources — even if someone later adds a permissive policy to the Spark role, the boundary blocks it.

Step 4 — Account C: Bucket Policy (allow Account D read-only)

```
{
  "Statement": [
    {
      "Sid": "AllowAccountDRead",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT_D:role/analyst-role"
      },
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::results-bucket",
        "arn:aws:s3:::results-bucket/*"
      ]
    }
  ]
}
```

Step 5 — Encryption re-keying (Account B writes to Account C with Account C's KMS key)

Account B's Spark job must encrypt output with Account C's KMS key. This requires:

1. Account C's KMS key policy allows Account B to use it for `GenerateDataKey`.
2. Account B's Spark role policy allows `kms:GenerateDataKey` on Account C's key.

When writing to S3:

```
s3.put_object(
    Bucket='results-bucket',
    Key='output/data.parquet',
    Body=data,
    ServerSideEncryption='aws:kms',
    SSEKMSKeyId='arn:aws:kms:ap-south-1:ACCOUNT_C:key/key-id-c'
)
```

Step 6 — Audit trail

- Enable **S3 Server Access Logging** or **S3 CloudTrail Data Events** on all three buckets.
- **CloudTrail** in each account with **AWS Organization-level CloudTrail** (all API calls logged centrally).
- KMS CloudTrail events automatically log every Decrypt/Encrypt call with the principal ARN.

Why This Approach

Requirement	Mechanism	Reason

Requirement	Mechanism	Reason
Account A inaccessible to Account D	No entry in Account A's bucket policy for Account D	Explicit allow required; default is deny
Spark reads Account A	Bucket policy + IAM role policy dual allow	Both resource-based and identity-based must allow
Re-encryption per account	KMS key per account, explicit GenerateDataKey grant	Data is encrypted with owner account's key
Spark can't access Account D	Permission boundary on Spark role	Boundary enforced regardless of policy additions
Audit trail	CloudTrail + S3 data events + KMS logs	All access, at all accounts, centrally queryable

Alternatives Considered

S3 replication (cross-account) instead of direct bucket policy access:

- Replication creates a full copy of data in Account B — violates the principle of least privilege and creates data sprawl.
- Direct bucket policy access means Account B reads data without copying it.

AWS Lake Formation for cross-account data access:

- Lake Formation provides fine-grained table/column-level access control over S3 data.
- Overkill for this scenario but worth mentioning for large-scale data mesh architectures.
- Lake Formation + Glue Data Catalog is the right answer when you have 50+ tables and need column-level security.

Centralised KMS key for all accounts:

- Simpler key management but violates account-level security isolation.
- If one account's admin is compromised, they could use the shared key to decrypt data across accounts.
- Per-account CMKs provide blast-radius limitation.

Pitfalls / Mistakes

- **The dual-allow requirement is the #1 miss in cross-account S3 access.** The bucket policy (resource side) must allow the cross-account role AND the IAM role (identity side) must allow the S3 action. Candidates always forget one side.
- **Forgetting the KMS cross-account grant.** S3 access working but KMS failing results in `AccessDenied` at object decryption — not at the S3 API call. The error message points to KMS, not S3, which confuses candidates.
- **Using the bucket owner's KMS key for writes from another account.** If Account B writes to Account C's bucket without specifying Account C's KMS key, the object may be encrypted with Account B's key — making it unreadable to Account C's applications.

- **Not using permission boundaries.** Without boundaries, an Account B admin with IAM permissions could attach a new policy to the Spark role granting it access to Account D. Boundaries provide a hard guard rail.

Scenario 8 — Microservices Across Accounts Communicating Without Internet Exposure

Scenario

Your company uses a multi-account strategy with AWS Organizations. Team Alpha runs an **order-service** on EKS in **Account A**. Team Beta runs a **payment-service** on EKS in **Account B**. The order-service calls the payment-service over HTTPS. Both services are in `ap-south-1`.

Constraints:

- Traffic must never touch the internet — all communication must be private.
- Account B's VPC must not be fully exposed to Account A (only payment-service should be accessible).
- Each service team manages their own account independently — no shared VPC.
- The payment-service may move between EKS node groups or scale to different IPs — the connection mechanism must not rely on fixed IPs.
- mTLS (mutual TLS) must be enforced between services.
- Latency must be < 2ms for the service call itself (network overhead only).

Solution

AWS PrivateLink + AWS Private CA (ACM PCA) for mTLS + Kubernetes ExternalName Service for DNS-based service discovery.

Architecture:

```
Account A (Order Service)                Account B (Payment Service)
┌──────────────────────────┐             ┌──────────────────────────────┐
│  EKS Cluster             │             │  EKS Cluster                 │
│  ┌────────────────────┐  │             │  ┌──────────────────────────┐  │
│  │  order-service pod │  │             │  │  payment-service pod     │  │
│  │  (mTLS client)     │──┼             ├──│  (mTLS server)           │  │
│  └────────────────────┘  │             │  └──────────────────────────┘  │
│          │               │             │              │                 │
│  ┌───────▼────────────┐  │             │  ┌───────────▼──────────────┐  │
│  │  Interface Endpoint │ │             │  │  NLB (payment-nlb)       │  │
│  │  (vpce-payment-svc) │─┼─PrivateLink│  TCP 8443                  │  │
│  │  DNS aliased:       │ │ │           │  └──────────────────────┘  │
│  │  payment.internal   │ │ │           │                              │
│  └────────────────────┘  │             │  VPC Endpoint Service        │
└──────────────────────────┘             │  (com.amazonaws.vpce.xxx)    │
                                         └──────────────────────────────┘
```

```
Both services share certificates from a common AWS Private CA
(or separate CAs with cross-account trust)
```

Step 1 — Account B: Expose payment-service via PrivateLink

The payment-service pods are fronted by a Kubernetes `LoadBalancer` service that creates an NLB in AWS:

```yaml
# payment-service/k8s/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: payment-service
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
    service.beta.kubernetes.io/aws-load-balancer-internal: "true"
    service.beta.kubernetes.io/aws-load-balancer-cross-zone-load-
balancing-enabled: "true"
spec:
  type: LoadBalancer
  ports:
  - port: 8443
    targetPort: 8443
    protocol: TCP
  selector:
    app: payment-service
```

Register this NLB as a VPC Endpoint Service (same as Scenario 1, Step 1–2).

Step 2 — Account A: Create Interface VPC Endpoint and set DNS alias

```bash
# Create endpoint
aws ec2 create-vpc-endpoint \
  --vpc-endpoint-type Interface \
  --service-name com.amazonaws.vpce.ap-south-1.vpce-svc-payment \
  --vpc-id vpc-account-a \
  --subnet-ids subnet-private-1a subnet-private-1b \
  --security-group-ids sg-payment-endpoint

# The endpoint gets a DNS name:
# vpce-xxx.vpce-svc-xxx.ap-south-1.vpce.amazonaws.com
```

Create a **Route 53 Private Hosted Zone** in Account A:

```
payment.internal.company.com → ALIAS → vpce-xxx.vpce-svc-xxx.ap-south-
1.vpce.amazonaws.com
```

Now the order-service calls `https://payment.internal.company.com:8443` — DNS-based, not IP-based.

Step 3 — mTLS with AWS Private CA

Both services need certificates from a trusted CA. Use **AWS Certificate Manager Private CA (ACM PCA)**:

Option A — Shared Private CA (simpler):

- Create one Private CA in a shared services account.
- Both Account A and Account B issue certificates from this CA.
- Each service presents its certificate; the other verifies it against the shared CA.

Option B — Cross-account CA trust (more isolated):

- Account A has its own Private CA.
- Account B has its own Private CA.
- Exchange CA certificates and configure cross-trust in each service's TLS config.

For this scenario, **Option A** (shared Private CA) is simpler and appropriate for a single organisation.

```
# Create Private CA in Shared Account
aws acm-pca create-certificate-authority \
  --certificate-authority-configuration \
    "KeyAlgorithm=RSA_2048,SigningAlgorithm=SHA256WITHRSA,
     Subject=
{Country=IN,Organization=MyCompany,CommonName=internal.company.com}" \
  --certificate-authority-type SUBORDINATE

# Issue cert for payment-service (Account B)
aws acm-pca issue-certificate \
  --certificate-authority-arn arn:aws:acm-pca:ap-south-
1:SHARED_ACCT:certificate-authority/xxx \
  --csr file://payment-service.csr \
  --signing-algorithm SHA256WITHRSA \
  --validity Value=365,Type=DAYS
```

Kubernetes sidecar for mTLS (using AWS App Mesh or Istio with SPIFFE/SPIRE):

For service mesh mTLS at scale, use **AWS App Mesh** with Envoy sidecars, or **Istio** with cert-manager + ACM PCA integration:

```
# Envoy sidecar handles mTLS transparently
# order-service pod → Envoy (mTLS) → PrivateLink Endpoint → Envoy
(payment-service) → payment-service pod
```

The Envoy sidecar handles certificate rotation, mTLS handshake, and retries — the application code doesn't need to manage TLS.

Step 4 — Security Group on the VPC Endpoint

```
Endpoint SG (Account A):
  Inbound:
    Port: 8443, Source: sg-order-service-pods
  Outbound:
    Port: 8443, Destination: (NLB in Account B via PrivateLink)
```

Why This Approach

Requirement	Component	Reason
No internet	PrivateLink	Traffic stays on AWS backbone
Service-level isolation	PrivateLink NLB (not full VPC)	Only payment-service port exposed
No fixed IPs	DNS alias to VPC Endpoint DNS	Endpoint DNS stable even if NLB IPs change
mTLS	ACM PCA + Envoy/App Mesh	Certificate-based mutual auth
<2ms network overhead	Same-region PrivateLink	Sub-millisecond overhead
Independent teams	Cross-account PrivateLink	Each account fully independent

---

Alternatives Considered

VPC Peering + internal DNS:

- Full VPC exposure — violates service-level isolation requirement.
- Requires CIDR coordination. If teams independently chose 10.0.0.0/16 for both VPCs, peering fails.

API Gateway (private) as intermediary:

- Adds ~5ms latency per call for API GW processing.
- Violates <2ms network latency requirement.
- Suitable for human-scale API traffic, not service-mesh sub-millisecond service calls.

AWS App Mesh cross-account (no PrivateLink):

- App Mesh doesn't provide private network-layer isolation between accounts by itself.
- Still needs PrivateLink or VPC Peering for the underlying network path.

Pitfalls / Mistakes

- **Relying on NLB IP addresses instead of DNS.** NLB IPs can change (especially with cross-zone load balancing and AZ changes). The PrivateLink endpoint DNS name is stable. Always use DNS.

- **Forgetting mTLS certificate renewal automation.** Certificates expire. Use ACM PCA's managed renewal or cert-manager with automated rotation. A certificate expiry in production causing service outage is embarrassing and preventable.
- **Not testing PrivateLink connectivity before service mesh setup.** Validate raw TCP connectivity through the PrivateLink endpoint before layering mTLS on top. mTLS failures and connectivity failures look similar at the application layer.
- **NLB health check protocol mismatch.** NLB health checks default to TCP. If you switch to HTTPS health checks (for mTLS validation), the NLB needs the certificate. Misconfigured health checks mark all targets unhealthy, causing PrivateLink to return connection refused.

Scenario 9 — Direct Connect Failover to VPN Without Dropping Sessions

Scenario

A large bank has a 10 Gbps AWS Direct Connect connection from their primary data centre to `ap-south-1`. They also have a Site-to-Site VPN over the internet as a backup. The bank's core banking application maintains **long-lived TCP sessions** (some lasting 30+ minutes) between on-premises application servers and RDS instances in the VPC.

During a scheduled DX maintenance window (1 hour), traffic must fail over to VPN **automatically** and then fail back to DX when maintenance ends — **without dropping existing TCP sessions**.

Constraints:

- TCP session continuity is mandatory (dropping sessions causes transaction rollbacks).
- Failover must complete within 30 seconds of DX failure.
- Fail-back to DX must be automatic and graceful (not disrupt sessions re-established on VPN).
- BGP must manage path preference.
- The VPN cannot handle 10 Gbps — it's limited to 1.25 Gbps. During the failover window, traffic must be throttled.

Solution

BGP-based path preference with BFD (Bidirectional Forwarding Detection) on DX + ECMP-capable VPN for throughput + AWS VGW with both DX and VPN attached.

Architecture:

```
On-Premises
┌──────────────────────────────────────────┐
│  Core Router (BGP ASN: 65000)            │
│                                          │
│  eBGP Session 1 → DX Virtual Interface   │  Primary (Local Pref 200)
│  eBGP Session 2 → VPN Customer GW        │  Backup  (Local Pref 100, AS
prepend 3x)
│                                          │
│  BFD enabled on DX BGP session           │
```

```
  │  BFD timers: tx=300ms, rx=300ms, mult=3 │
  │  (failure detection: ~900ms)            │
  └─────────────────────────────────────────┘
            │                    │
            │ DX (10Gbps)        │ VPN (IPSec, 1.25Gbps per tunnel)
            ▼                    ▼
     Virtual Private Gateway (ap-south-1)
            │
            ▼
     VPC (10.0.0.0/16)
     RDS instances, App tier
```

BGP configuration for path preference:

```
# On-premises router config (Cisco IOS-style pseudocode)

# DX path: highest local preference (most preferred)
route-map DX-IN permit 10
  set local-preference 200

# VPN path: low local preference + AS path prepending (least preferred)
route-map VPN-IN permit 10
  set local-preference 100
  set as-path prepend 65000 65000 65000   # Makes VPN less preferred from
AWS → on-prem direction

neighbor DX-BGP-PEER route-map DX-IN in
neighbor VPN-BGP-PEER route-map VPN-IN in
```

BFD for sub-second failure detection:

Standard BGP keepalive/hold timers (30s keepalive, 90s hold) would cause 90-second failover — too slow. **BFD (Bidirectional Forwarding Detection)** is a separate lightweight hello protocol that runs independently of BGP:

```
BFD timers: 300ms transmit, 300ms receive, multiplier 3
→ Failure detection: 3 × 300ms = ~900ms
→ BGP then reconverges immediately (BFD notifies BGP of peer failure)
→ Total failover: < 3 seconds (900ms detection + ~1–2s BGP convergence)
```

AWS Direct Connect supports BFD — enable it on the DX Virtual Interface.

TCP Session Continuity During Failover:

This is the hardest part. TCP sessions survive network path changes **only if the source/destination IPs remain constant**. Since both DX and VPN terminate on the same VGW (and the VGW presents the same private IP to the VPC), the VPC side doesn't change. The on-premises side also doesn't change (application servers keep their IPs).

The challenge is the **routing path change itself** — a brief period during BGP reconvergence where packets may be lost. Configuring BFD ensures this window is < 3 seconds.

For the application to survive this:

1. **TCP keepalive tuning:** Set OS-level TCP keepalives to detect and handle brief interruptions gracefully.
2. **Application-layer reconnection logic:** RDS client libraries (e.g., JDBC, psycopg2) should have retry logic for transient connection errors.
3. **Use RDS Proxy:** RDS Proxy maintains persistent connections to the database from within the VPC. When on-prem application connections drop during failover, they reconnect to RDS Proxy — which maintains the database connection pool, reducing reconnection time to < 1 second.

VPN throughput limitation during failover:

The bank's 10Gbps DX carries the full production load. VPN is limited to 1.25Gbps. During the DX maintenance window:

1. Use **AWS VPN Accelerated** (Global Accelerator-backed VPN) for improved throughput — up to 4Gbps with ECMP across 2 VPN tunnels.
2. **Traffic throttling on-prem:** Use QoS to prioritise critical transaction traffic (core banking) over batch/reporting traffic during the maintenance window.
3. **Schedule maintenance during off-peak:** Work with the DX provider to schedule maintenance at 2am when throughput is ~500Mbps — well within VPN capacity.

Fail-back to DX (graceful):

When DX comes back online:

1. BFD detects DX peer restoration.
2. BGP re-establishes and re-advertises routes with Local Pref 200 (DX preferred).
3. New connections automatically use DX.
4. **Existing sessions on VPN continue on VPN** until they naturally terminate — don't forcefully cut them over (this would drop sessions).
5. Over 5–10 minutes, as sessions on VPN naturally close and reopen, they establish over DX.

This is **graceful fail-back** — no forced session termination.

Why This Approach

Requirement	Mechanism	Reason
<30s failover	BFD + BGP (not just BGP timers)	BFD detects in ~900ms; BGP timers alone = 90s
No session drop	VGW same IP for both paths; graceful fail-back	Path changes with same endpoints; TCP survives if packets lost < TCP timeout
Automatic path preference	BGP Local Preference + AS prepend	Standards-based; works with any router vendor

Requirement	Mechanism	Reason
Throughput management	ECMP VPN + off-peak scheduling + QoS	VPN can't match DX; manage during window

Alternatives Considered

DX resilient/redundant connections (second DX link):

- Adding a second DX connection (to a different DX location and PoP) eliminates the maintenance window problem entirely.
- Most cost-effective long-term solution for production banking workloads.
- Two DX connections at 10Gbps each with different PoPs and on-prem routers = true HA.
- VPN becomes a tertiary fallback.

Restoring DX before fail-back (manual intervention):

- Manual fail-back requires operational procedures and on-call engineers.
- BGP-driven automatic fail-back is preferable for a 1-hour maintenance window.

Pitfalls / Mistakes

- **Not enabling BFD on DX.** Without BFD, BGP hold-down timer (default 90 seconds) means 90-second failover — twice the allowed window. BFD is the answer to fast DX failure detection.
- **Assuming TCP sessions will survive a 3-second outage.** TCP's default retransmission timeout is 1–3 minutes — a 3-second packet loss does NOT drop the TCP session. Most applications survive this window. Candidates often assume they need zero-packet-loss failover.
- **Forgetting VPN throughput limits.** A 10Gbps DX workload cannot failover to a standard VPN. This must be in the design consideration — either capped throughput or accelerated VPN with ECMP.
- **Not testing failover in a maintenance window before production.** Every DR mechanism must be fire-drilled. BGP reconvergence, BFD timers, and RDS connection retry behaviour must be validated before a real failure — not during one.

Scenario 10 — Multi-Account Centralised Egress Inspection Architecture

Scenario

Your company has 30+ AWS accounts under an Organization. The security team mandates that **all internet-bound traffic from all accounts** must pass through a centralised **Network Firewall** for inspection, before reaching the internet. Individual accounts must NOT have their own internet egress path.

Constraints:

- New accounts added to the org must automatically inherit the egress path.
- Spoke accounts must not be able to bypass the centralised firewall.
- The firewall must inspect TLS-encrypted traffic (SSL decryption required for HTTPS flows).
- The firewall VPC must not allow any inbound traffic from the internet to spoke VPCs.

- High availability — the firewall must survive an AZ failure.

Solution

Centralised Egress VPC with AWS Network Firewall + AWS Transit Gateway + Service Control Policies (SCPs) to prevent bypass.

Architecture:

```
                          Transit Gateway
                    ┌──────────────────────────────────┐
                    │  TGW Route Tables:               │
                    │  SPOKE-RT: 0.0.0.0/0 → Egress│
                    │  EGRESS-RT: propagate spokes  │
                    └──────────────────────────────────┘
                       │ (Spoke attachments)  │ (Egress VPC attachment)
           ┌───────────┘                      └──────────────────┐
           │  Spoke VPCs (30+ accounts)                          │
           │  No IGW, no NAT                                     ▼
           │  0.0.0.0/0 → TGW                        Egress VPC (Security
Account)
           │
┌──────────────────────────────┐
           │                                          │  AZ-a           AZ-
b          │
           │                                          │  ┌────────────┐
┌──────────┐ │
           │                                          │  │ Firewall  │ │
Firewall  │ │
           │                                          │  │ Endpoint  │ │
Endpoint  │ │
           │                                          │  └─────┬──────┘
└────┬─────┘ │
           │                                          │        │
│        │
           │                                          │
┌───▼──────────────▼──────┐ │
           │                                          │  │  AWS Network
Firewall    │ │
           │                                          │  │  (Stateful +
Stateless)  │ │
           │                                          │  │  + TLS
Inspection Config │ │
           │                                          │
└────────────────────────┘ │
           │                                          │        │
│
           │                                          │
┌───▼──────────────────┐  │
           │                                          │  │  NAT Gateway
(per AZ)   │  │
```

```
          │                                                │
└──────┬──────────────────────┘  │
          │                                                │        │
│
          │                                                │
┌──────▼──────────────────┐  │
          └────────────────────────────────────────────┘  │  Internet
Gateway      │  │

└──────────────────────┘  │

└────────────────────────┘
```

Step 1 — TGW Route Table Configuration

```
SPOKE-RT (associated with all spoke VPC attachments):
  0.0.0.0/0 → Egress VPC TGW attachment
  (Spoke VPCs have no other route to internet)

EGRESS-RT (associated with Egress VPC attachment):
  Propagate all spoke CIDR routes (for return traffic)
  0.0.0.0/0 → (not needed; NAT GW handles internet return path)
```

Step 2 — Egress VPC Routing (the "bump-in-the-wire" pattern)

Inside the Egress VPC, packets must flow: TGW → Firewall → NAT → Internet (and reverse).

This requires careful routing across subnets:

```
Subnet: TGW-Attachment-Subnet (one per AZ)
Route Table:
  0.0.0.0/0 → Firewall Endpoint (AZ-local)

Subnet: Firewall-Subnet (one per AZ)
Route Table:
  0.0.0.0/0 → NAT Gateway (AZ-local)

Subnet: NAT-Public-Subnet (one per AZ)
Route Table:
  0.0.0.0/0 → Internet Gateway
  10.0.0.0/8 → TGW attachment (return traffic to spokes)

Firewall Endpoint routing:
  For inbound (internet → VPC):
    Route to TGW attachment subnet
  For outbound (VPC → internet):
    Already handled by Firewall-Subnet route to NAT
```

Step 3 — AWS Network Firewall with TLS Inspection

```
aws network-firewall create-firewall \
  --firewall-name centralised-egress-firewall \
  --firewall-policy-arn arn:aws:network-firewall:ap-south-
1:SECURITY_ACCOUNT:firewall-policy/egress-policy \
  --vpc-id vpc-egress \
  --subnet-mappings SubnetId=subnet-az-a SubnetId=subnet-az-b \
  --delete-protection \
  --firewall-policy-change-protection
```

TLS Inspection Configuration (for HTTPS inspection):

```
aws network-firewall create-tls-inspection-configuration \
  --tls-inspection-configuration-name egress-tls-inspection \
  --tls-inspection-configuration \
    "ServerCertificateConfigurations=[{
      Scopes=[{
        Protocols=[6],
        DestinationPorts=[{FromPort=443,ToPort=443}],
        Sources=[{AddressDefinition=0.0.0.0/0}],
        Destinations=[{AddressDefinition=0.0.0.0/0}]
      }],
      ServerCertificates=[{ResourceArn=arn:aws:acm:...:certificate/xxx}],
      CertificateAuthorityArn=arn:aws:acm-pca:...:certificate-
authority/xxx
    }]"
```

Network Firewall acts as a TLS proxy — it intercepts the HTTPS connection, decrypts it using a re-signing CA certificate, inspects the payload, then re-encrypts with the original destination certificate. Client workloads must trust the Network Firewall's CA certificate.

Stateful Rule Groups (examples):

```
{
  "RuleGroupName": "egress-domain-allowlist",
  "Type": "STATEFUL",
  "Rules": "pass tls $HOME_NET any -> $EXTERNAL_NET 443 (tls.sni;
content:\"amazonaws.com\"; msg:\"Allow AWS services\"; sid:1;)\npass tls
$HOME_NET any -> $EXTERNAL_NET 443 (tls.sni; content:\"github.com\";
msg:\"Allow GitHub\"; sid:2;)\ndrop tls $HOME_NET any -> $EXTERNAL_NET 443
(msg:\"Block all other HTTPS\"; sid:9999;)"
}
```

Step 4 — SCPs to prevent bypass (the enforcement layer)

SCPs at the AWS Organization level prevent spoke accounts from creating their own IGW or VPN:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyInternetGatewayCreation",
      "Effect": "Deny",
      "Action": [
        "ec2:CreateInternetGateway",
        "ec2:AttachInternetGateway",
        "ec2:CreateNatGateway",
        "ec2:CreateVpnGateway",
        "ec2:CreateVpnConnection"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:PrincipalOrgID": "SECURITY_TEAM_OU_ID"
        }
      }
    }
  ]
}
```

Apply this SCP to all OUs except the Security OU. This is the **guardrail** — even if a developer tries to create a NAT Gateway to bypass the firewall, the SCP denies it.

Why This Approach

Requirement	Component	Reason
All egress via firewall	TGW SPOKE-RT 0.0.0.0/0 → Egress VPC	Centralised routing; no local IGW
New accounts inherit egress	TGW RAM-shared + SPOKE-RT default	Auto-attach; default route pre-set
Cannot bypass	SCPs deny IGW/NAT creation in spoke accounts	Policy enforced at org level
TLS inspection	Network Firewall TLS Inspection Config	Decrypt-inspect-re-encrypt in-line
AZ HA	Firewall endpoints per AZ + AZ-local routing	AZ failure doesn't affect other AZ

Pitfalls / Mistakes

- **Asymmetric routing in the Egress VPC.** Getting the routing correct inside the Egress VPC (TGW subnet → Firewall → NAT → IGW, and the exact reverse) is the hardest part of this architecture. One

wrong route and traffic loops or drops silently.

- **Forgetting the SCP.** The firewall is technical enforcement. Without the SCP, a determined developer with sufficient IAM permissions in their spoke account creates an IGW and bypasses the firewall entirely. Architecture + policy enforcement together = real security.
- **TLS inspection breaks certificate pinning.** Any application that pins certificates (checks the exact certificate fingerprint) will fail TLS inspection because the firewall re-signs with its own CA. Identify certificate-pinning applications and either exempt them or deploy their certificates on the firewall's trust chain.
- **Not making firewall endpoints AZ-local.** If all AZs route through one firewall endpoint, a single AZ failure disrupts the entire egress path. Route each AZ's traffic to that AZ's firewall endpoint independently.

Scenario 11 — Database in a Locked-Down Account Accessed by Lambda in Another Account

Scenario

Your company's security policy states that the RDS database (containing PII) must live in a dedicated **Database Account** with no public internet access and no VPC peering to general-purpose accounts. Your **Application Account** has Lambda functions that need to query this database. Both accounts are in the same AWS Organisation in `ap-south-1`.

Constraints:

- The Database Account must have no internet access whatsoever.
- Lambda in the Application Account is VPC-attached (runs inside a VPC).
- Connections to RDS must use IAM database authentication (no long-lived passwords).
- RDS must use **RDS Proxy** to handle Lambda's spiky connection patterns.
- The Lambda function must NOT be able to access any other resource in the Database Account — only the RDS Proxy endpoint.
- Latency for Lambda → DB call must be < 10ms network overhead.

Solution

PrivateLink (VPC Endpoint Service) exposing RDS Proxy from Database Account to Application Account + IAM Auth via STS cross-account role assumption.

Architecture:

```
Application Account                    Database Account
┌──────────────────────────┐
┌──────────────────────────────────────┐
│  Lambda (VPC-attached)   │          │  VPC (10.2.0.0/16) — NO IGW      │
│  VPC: 10.1.0.0/16        │          │
│
│  ┌────────────────────┐  │          │  ┌──────────────────────────────┐
│
```

```
│  │  Lambda Function    │ │          │  │  RDS Proxy                 │
│
│  │  - Assumes DB-Access│─┤          ├──│  (MySQL/Postgres)          │
│
│  │    Role (cross-acct)│ │PrivateLink│  │  - IAM auth enabled       │
│
│  └──────────┬──────────┘ │──────────│  └────────────────────────────┘    │
│             │             │          │              │
│
│  ┌──────────▼──────────┐ │          │  ┌──────────────▼──────────────┐  │
│  │  Interface Endpoint │ │          │  │  NLB (proxy-nlb) TCP 5432   │  │
│  │  (vpce-db-proxy)    │ │          │  └─────────────────────────────┘  │
│  └─────────────────────┘ │          │
│
└──────────────────────────┘          │  VPC Endpoint Service
│
                                      │  (Allows only Application Acct)  │

└──────────────────────────────────┘
```

Step 1 — Database Account: Set up RDS Proxy with IAM authentication

```
aws rds create-db-proxy \
  --db-proxy-name pii-db-proxy \
  --engine-family POSTGRESQL \
  --auth
'[{"AuthScheme":"SECRETS","IAMAuth":"REQUIRED","SecretArn":"arn:aws:secret
smanager:..."}]' \
  --role-arn arn:aws:iam::DB_ACCOUNT:role/rds-proxy-role \
  --vpc-subnet-ids subnet-db-a subnet-db-b \
  --vpc-security-group-ids sg-rds-proxy \
  --require-tls true
```

`"IAMAuth":"REQUIRED"` means only IAM-authenticated connections are accepted — no password-based auth possible.

Step 2 — Database Account: Expose RDS Proxy via PrivateLink

Create an NLB pointing to the RDS Proxy endpoint:

```
# NLB target: RDS Proxy endpoint hostname (resolve to IPs for IP-type
target group)
# NLB listener: TCP 5432 (PostgreSQL) or 3306 (MySQL)
```

Register as VPC Endpoint Service:

```
aws ec2 create-vpc-endpoint-service-configuration \
  --network-load-balancer-arns
arn:aws:elasticloadbalancing:...:loadbalancer/net/proxy-nlb/xxx \
  --acceptance-required true

# Allow only Application Account
aws ec2 modify-vpc-endpoint-service-permissions \
  --service-id vpce-svc-xxx \
  --add-allowed-principals arn:aws:iam::APP_ACCOUNT:role/lambda-db-role
```

Step 3 — Application Account: Lambda role + cross-account IAM for RDS IAM auth

RDS IAM authentication requires a token generated by STS. For cross-account access:

1. Lambda assumes a role in the Database Account that has `rds-db:connect` permission.

   Database Account: Create DB-Access role:

   ```
   {
     "Trust Policy": {
       "Principal": {"AWS": "arn:aws:iam::APP_ACCOUNT:role/lambda-db-
   role"},
       "Action": "sts:AssumeRole"
     },
     "Permission Policy": {
       "Action": "rds-db:connect",
       "Resource": "arn:aws:rds-db:ap-south-1:DB_ACCOUNT:dbuser:prx-
   xxx/app_user"
     }
   }
   ```

2. **Application Account: Lambda function code:**

   ```
   import boto3
   import psycopg2

   def get_rds_token():
       # Assume cross-account role
       sts = boto3.client('sts')
       creds = sts.assume_role(
           RoleArn='arn:aws:iam::DB_ACCOUNT:role/db-access-role',
           RoleSessionName='lambda-session'
       )['Credentials']

       # Use assumed role credentials to generate RDS IAM token
       rds_client = boto3.client(
           'rds',
           region_name='ap-south-1',
           aws_access_key_id=creds['AccessKeyId'],
   ```

```
        aws_secret_access_key=creds['SecretAccessKey'],
        aws_session_token=creds['SessionToken']
    )

    token = rds_client.generate_db_auth_token(
        DBHostname='vpce-xxx.vpce-svc-xxx.ap-south-
1.vpce.amazonaws.com',
        Port=5432,
        DBUsername='app_user'
    )
    return token, creds

def handler(event, context):
    token, _ = get_rds_token()
    conn = psycopg2.connect(
        host='vpce-xxx.vpce-svc-xxx.ap-south-1.vpce.amazonaws.com',
        port=5432,
        database='appdb',
        user='app_user',
        password=token,
        sslmode='require'
    )
    # Execute query...
```

Step 4 — Lambda VPC Configuration

Lambda must be VPC-attached to use a VPC Endpoint:

```
aws lambda update-function-configuration \
  --function-name pii-query-function \
  --vpc-config SubnetIds=subnet-lambda-a,subnet-lambda-
b,SecurityGroupIds=sg-lambda
```

The Lambda's security group must allow outbound to the VPC Endpoint security group on port 5432.

Step 5 — Connection pooling (RDS Proxy handles Lambda's connection bursts)

Lambda can have thousands of concurrent executions, each opening a new DB connection. Without RDS Proxy, this exhausts PostgreSQL's `max_connections`. RDS Proxy pools connections:

- Lambda → RDS Proxy: up to thousands of connections.
- RDS Proxy → RDS: limited, pooled connections (e.g., 100 to the actual DB instance).

RDS Proxy is in the Database Account's VPC — traffic from Lambda flows via PrivateLink to RDS Proxy, which then connects to RDS internally within the Database Account VPC.

Why This Approach

Requirement	Component	Reason

Requirement	Component	Reason
No internet in DB account	No IGW/NAT; PrivateLink only ingress	PrivateLink doesn't require internet
Only RDS Proxy exposed	PrivateLink exposes single NLB endpoint	Not the whole VPC; service-level isolation
IAM auth, no passwords	RDS Proxy IAM auth required	Eliminates credential management
Lambda connection bursts	RDS Proxy pooling	Prevents DB connection exhaustion
Cross-account IAM auth	STS AssumeRole + rds-db:connect	Standard AWS cross-account pattern

Pitfalls / Mistakes

- **Generating IAM auth tokens with the Lambda's own account role instead of the cross-account role.** The token must be generated using credentials that have `rds-db:connect` on the DB account's proxy resource. Using the Lambda's own role generates a token that the DB account's RDS Proxy won't accept.
- **RDS Proxy endpoint DNS not resolving in Lambda's VPC.** The VPC Endpoint's private DNS must be enabled and the Lambda's VPC must have `enableDnsHostnames` and `enableDnsSupport` set to `true`. Without this, the hostname doesn't resolve.
- **Not accounting for RDS Proxy warm-up time.** The first connection through RDS Proxy after a cold period takes ~1–2 seconds to establish. Lambda cold starts + RDS Proxy warm-up = slow first invocation. Use Lambda provisioned concurrency for latency-sensitive functions.
- **PrivateLink NLB pointing to RDS Proxy DNS name (not IP) in target group.** NLB IP-type target groups need IPs, not hostnames. You must resolve the RDS Proxy endpoint to IPs and register them. Or use an ALB (which supports hostname targets) — but ALB doesn't work with PrivateLink. Solution: use a Lambda-based target registration that periodically resolves RDS Proxy IPs and updates the NLB target group.

Scenario 12 — Active-Passive DR Across Regions with Sub-5-Minute RTO

Scenario

A healthcare platform runs in `ap-south-1` (Mumbai, primary) and needs a DR site in `ap-southeast-1` (Singapore). The system has:

- ECS-based application tier (50 services).
- Aurora PostgreSQL database (500 GB).
- ElastiCache Redis cluster (session data, 10 GB).
- S3 buckets (patient documents, 5 TB).
- Route 53 for DNS.

DR Requirements:

- **RTO (Recovery Time Objective): < 5 minutes.**
- **RPO (Recovery Point Objective): < 30 seconds** (for database; eventual for S3/Redis).
- The DR site must be **warm** (pre-provisioned infrastructure) but not serving production traffic during normal operations.
- Failover must be initiated manually by an on-call engineer (one-click, but human-approved).
- No data must be permanently written to `ap-southeast-1` during normal operations (data residency).

Solution

Pilot Light DR architecture with Aurora Global Database (cross-region replica), S3 Cross-Region Replication, pre-deployed ECS infrastructure in `ap-southeast-1` at reduced capacity, and Route 53 Health Check-gated DNS failover with manual approval gate.

Architecture — Normal State:

```
ap-south-1 (Active)                    ap-southeast-1 (Warm Standby)
┌──────────────────────────────────┐
┌──────────────────────────────────────┐
│  Route 53: 100% traffic          │      │  Route 53: 0% traffic (weight=0)
│                                  │      │
│  ECS: 50 services (full cap.)    │      │  ECS: 50 services (min capacity:
│
│  ALB (active)                    │      │  1 task per service — warm but
│                                  │      │
│                                  │      │  not serving traffic)
│
│  Aurora (Primary writer)         │──────►  Aurora Global DB (read-only
│
│  (500 GB, writes accepted)       │  ~1s │  secondary — no writes, RPO <30s)
│                                  │      │
│
│  ElastiCache Redis               │      │  ElastiCache Redis (empty —
│
│  (active, 10GB sessions)         │      │  will warm on failover)
│                                  │      │
│
│  S3 (patient docs, 5TB)          │──────►  S3 (CRR — eventual replication,
│
│  (active writes)                 │  CRR │  typically < 15 min for new objs)
│
└──────────────────────────────────┘
└──────────────────────────────────────┘
```

Failover Runbook (target: < 5 minutes total):

```
T+0:00  On-call engineer triggers DR failover via AWS Systems Manager
         Automation document (OpsItem auto-created by CloudWatch Alarm)

T+0:30  Automation Step 1: Promote Aurora Global Database secondary
         aws rds failover-global-cluster \
           --global-cluster-identifier healthcare-global-db \
           --target-db-cluster-identifier aurora-sg-cluster
         (Promotion: ~60 seconds for Aurora)

T+1:30  Automation Step 2: Update ECS services in ap-southeast-1 to
desired count
         Scale from 1 task → production capacity (e.g., 10 tasks per
service)
         ECS Service Update: desired-count=10 for all 50 services
         (ECS scaling: ~90 seconds for most services if images are in ECR
ap-southeast-1)

T+2:00  Automation Step 3: Verify ALB health in ap-southeast-1
         Wait for ALB target group: HealthyHostCount >= minimum threshold

T+3:00  Automation Step 4: Flip Route 53 DNS
         aws route53 change-resource-record-sets \
           --change-batch file://dr-failover-dns-change.json
         (TTL pre-set to 30s — propagates in ~30s)

T+3:30  Automation Step 5: Validate — synthetic monitoring check
         Run canary tests against ap-southeast-1 ALB endpoint
         If canary passes: proceed
         If canary fails: trigger rollback automation

T+4:30  DNS propagated globally. Traffic routing to ap-southeast-1.

T+5:00  Failover complete. RTO achieved.
```

Component-by-Component Design:

Aurora Global Database (RPO < 30 seconds):

```
# Create global cluster with primary in ap-south-1
aws rds create-global-cluster \
  --global-cluster-identifier healthcare-global-db \
  --source-db-cluster-identifier arn:aws:rds:ap-south-
1:ACCOUNT:cluster:aurora-mumbai

# Add ap-southeast-1 secondary
aws rds create-db-cluster \
  --db-cluster-identifier aurora-sg-cluster \
  --engine aurora-postgresql \
  --global-cluster-identifier healthcare-global-db \
  --region ap-southeast-1
```

Aurora Global DB replication lag: typically < 1 second. RPO of 30 seconds is easily achievable.

S3 Cross-Region Replication (S3 CRR):

```
aws s3api put-bucket-replication \
  --bucket patient-docs-mumbai \
  --replication-configuration '{
    "Role": "arn:aws:iam::ACCOUNT:role/s3-replication-role",
    "Rules": [{
      "Status": "Enabled",
      "Destination": {
        "Bucket": "arn:aws:s3:::patient-docs-singapore",
        "ReplicaKmsKeyID": "arn:aws:kms:ap-southeast-1:ACCOUNT:key/sg-
key",
        "StorageClass": "STANDARD"
      },
      "SourceSelectionCriteria": {
        "SseKmsEncryptedObjects": {"Status": "Enabled"}
      }
    }]
  }'
```

For S3 Replication Time Control (RTC), enable it to guarantee 99.99% of objects replicate within 15 minutes. This does not affect the < 5 min RTO (the app failover completes before S3 replication fully catches up for recent objects).

ECS Warm Standby (minimum capacity):

Run 1 task per ECS service in ap-southeast-1 at all times:

- Images are pulled from ECR (which is regional — images must be replicated to ap-southeast-1 ECR).
- Services can scale from 1 → 10 in 90 seconds with pre-warmed capacity reservations.
- ECS Capacity Provider with FARGATE and pre-provisioned ENI at minimum count.

Container image replication:

```
# Set up ECR replication rules
aws ecr put-replication-configuration \
  --replication-configuration '{
    "rules": [{
      "destinations": [{"region": "ap-southeast-1", "registryId":
"ACCOUNT"}],
      "repositoryFilters": [{"filter": "healthcare-", "filterType":
"PREFIX_MATCH"}]
    }]
  }'
```

Data Residency Consideration:

During normal operations, no patient data is written to `ap-southeast-1`. Aurora secondary is read-only. S3 CRR creates replicas but under a restricted bucket policy that prevents access except during an active DR event. The DR bucket policy includes a condition:

```
{
  "Condition": {
    "StringEquals": {
      "aws:PrincipalTag/dr-mode": "active"
    }
  }
}
```

During failover, the automation document attaches a tag to the ECS task role in ap-southeast-1 — unlocking S3 access.

Why This Approach

Requirement	Component	Reason
RTO < 5 min	SSM Automation + pre-warm ECS + pre-promoted Aurora	Automated, pre-staged infra; no provisioning during DR
RPO < 30s (DB)	Aurora Global Database	Storage-layer replication; ~1s typical lag
Human approval gate	SSM Automation with approval step	Human confirms before DNS flip
No data in standby during normal ops	Aurora read-only; S3 bucket policy gate	Data residency compliance
One-click failover	SSM Automation document	Single OpsItem approval triggers full runbook

---

Alternatives Considered

Cold Standby (no pre-provisioned infra):

- Provision ECS, Aurora (restore from snapshot), Redis on demand during DR.
- RTO: 30–60 minutes minimum. Does not meet 5-minute requirement.

Active-Active (both regions serving traffic):

- Achieves zero RTO — failover is transparent.
- Requires Aurora Global Database with fast failover promotion or DynamoDB Global Tables.
- Significantly higher cost (double the compute running at full capacity).
- Write consistency complexity (single Aurora writer means one region still handles writes).
- Over-engineered for a warm standby requirement unless budget justifies it.

Backup and Restore:

- RPO: hours (last backup). Completely misses the 30-second RPO requirement for RDS.
- Only appropriate for non-critical data tiers (archived records, reports).

Pitfalls / Mistakes

- **Not pre-replicating container images to the DR region.** If ECR images aren't in ap-southeast-1, ECS tasks during DR must pull from ap-south-1 over cross-region links — slow and unreliable when ap-south-1 may be degraded. Always use ECR replication or cross-region ECR pull-through cache.
- **Aurora promotion taking longer than expected.** Aurora Global DB promotion involves detaching the secondary from the global cluster and promoting it as an independent writer. If there's outstanding replication lag at failure time, promotion takes longer. Monitor `AuroraGlobalDBReplicationLag` — if it spikes, investigate before a real DR event.
- **DNS TTL not pre-set to low values.** If Route 53 records have a 300-second TTL and failover is triggered, it takes up to 5 minutes for DNS to propagate — using the entire RTO budget. Pre-set TTL to 30 seconds at least 24 hours before any expected DR event.
- **Forgetting Redis warm-up time.** After failover, the Redis cluster in ap-southeast-1 is empty — all sessions are gone. Users must re-authenticate. If this is unacceptable, implement cross-region Redis replication (ElastiCache Global Datastore) at the cost of replication complexity. Make an explicit product decision about session invalidation on DR.
- **Not fire-drilling the runbook.** DR runbooks that have never been tested fail at the worst possible moment. Schedule quarterly DR tests with actual traffic failover to ap-southeast-1 in a maintenance window.

End of AWS Advanced Architecture Interview Prep

> ## Revision Strategy
>
> **For cross-account questions:** Always think in terms of: trust policies, resource-based policies, and the dual-allow requirement. Both identity and resource policies must permit the action.
>
> **For networking questions:** Draw the packet's journey: source → security group → NACL → route table → target. For cross-region or hybrid: source → VPC → TGW/peering/DX → destination VPC → target.
>
> **For HA/DR questions:** Always clarify RTO vs RPO. State the trade-offs between cost and recovery objectives explicitly. Interviewers reward structured thinking over memorised answers.
>
> **For security boundary questions:** Identify what is being protected, from whom, and at which layer (network, IAM, data). Cover all three layers in your answer.
>
> **For hybrid connectivity:** BGP is the glue. Know Local Preference, MED, AS path prepending, and BFD. Interviewers at senior level expect you to know how routing decisions are made, not just that "Direct Connect and VPN are used together."